EC5207: DEVOPS ENGINEERING

ASSIGNMENT – CI/CD DIAGRAM

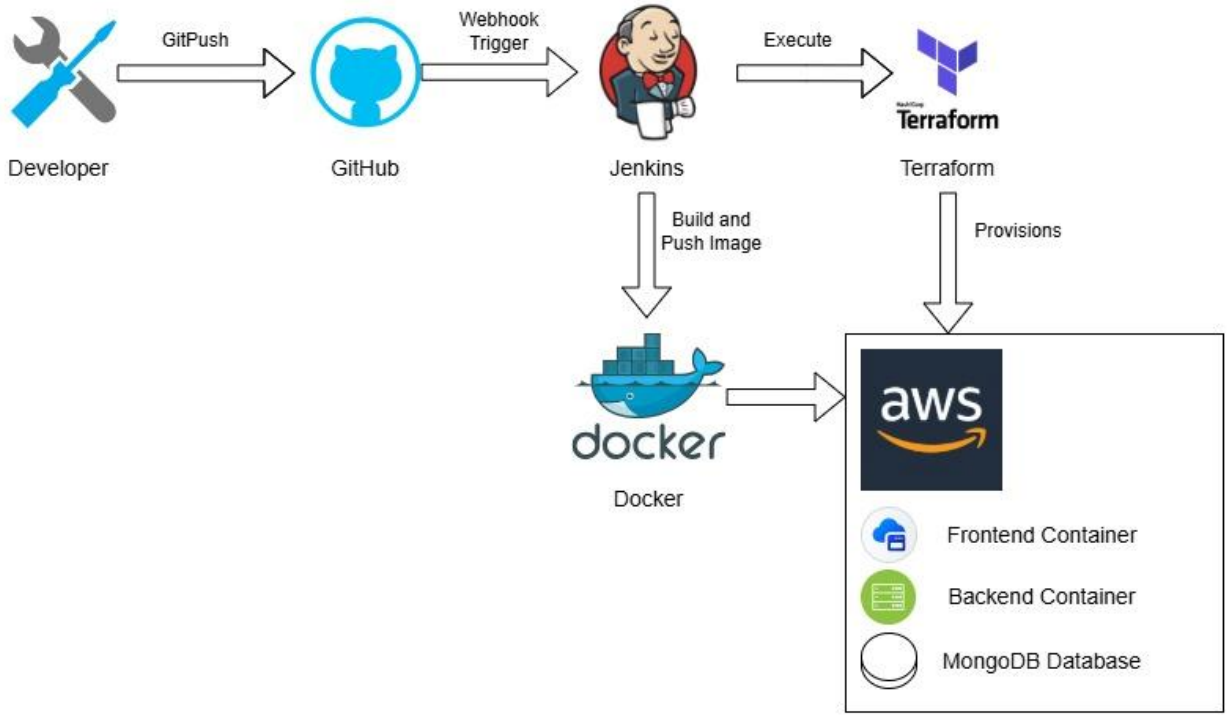
NAME :BORALUGODA. U.K.R.P.Y

REG.NO :EG/2022/4968

SEMESTER :05

DATE :22/01/2025

CI/CD DIGRAM



This diagram shows the complete path from developer to running the code in cloud. Starting with developer pushes to GitHub, which automatically triggers to Jenkins to take actions. Jenkins handle packaging the app into docker images for docker hub and using terraform to initiate aws server. Once all the configurations are ready, the server pulls the new images and launches the frontend, backend and database containers to get the application online.

AUTOMATION APPROACH

Tools

Git (2.43.0)	Version Control is the main purpose. Used to source code management
Jenkins (2.528.2)	CI/CD Automation: Manage the deployment of pipeline
Docker (28.4.0)	Packages the application and its dependencies into a container to maintain consistency.
Docker Hub	Stores the built Docker images to easily run on the deploy on server
Terraform (1.14.3)	Automates the provisioning and management of cloud infrastructure (AWS)
AWS	The cloud platform where the application containers are hosted and run.

Application Tools and Dependencies

JavaScript (24.11.1)	The core programming language.
Node.js	Backend
React.js	Frontend
MongoDB	NoSQL Database

Pipeline Stages

1. **Checkout SCM:** The pipeline is automatically triggered upon code commits. Jenkins retrieves the latest version of both the Spring Boot backend and React frontend from the source repository.

2. **Backend Build:** Installs Node.js dependencies using npm install and builds the backend application using npm run build, preparing it for containerization.

3. **Build Docker Images:** Dockerfiles are used to build container images for both backend and frontend services. Each image is tagged with a unique build identifier.

4. **Login to Docker Hub:** Jenkins authenticates with Docker Hub to gain permission to upload container images.
5. **Push Images to Docker Hub:** The generated Docker images are uploaded to Docker Hub

6. **Infrastructure Provisioning:** Plan by comparing current and desired AWS setups, then apply changes by creating or updating **EC2 instances** and security settings.

7. **Deployment (AWS):** The pipeline connects to the AWS EC2 instance via SSH, pulls the latest images from Docker Hub, and restarts the containers using docker-compose to apply the update.

8. **Clean Up:** Post-deployment, the pipeline removes temporary build artifacts and local Docker images to maintain a clean workspace.